Towards C++23 executors: A proposal for an initial set of algorithms

Document #:	P1897R0
Date:	2019-10-06
Project:	Programming Language C++ SG1

Reply-to:	Lee Howes <lwh@fb.com>

§ 1 Introduction

In [P0443R11] we have included the fundamental principles described in [P1660R0], and the fundamental requirement to customize algorithms. In recent discussions we have converged to an understanding of the `submit` operation on a `sender` acting as a fundamental interoperation primitive, and algorithm customization giving us full flexibility to optimize, to offload and to avoid synchronization in chains of mutually compatible algorithm customizations.

As a starting point, in [P0443R11] we only include a `bulk_execute` algorithm, that satisfies the core requirement we planned with P0443 to provide scalar and bulk execution. To make the C++23 solution completely practical, we should extend the set of algorithms, however. This paper suggests an expanded initial set that enables early useful work chains. This set is intended to act as a discussion focus for us to discuss one by one, and to analyze the finer constraints of the wording to make sure we do not over-constrain the design.

In the long run we expect to have a much wider set of algorithms, potentially covering the full set in the current C++20 parallel algorithms. The precise customization of these algorithms is open to discussion: they may be individually customized and individually defaulted, or they may be optionally individually customized but defaulted in a tree such that customizing one is known to accelerate dependencies. It is open to discussion how we achieve this and that is an independent topic, beyond the scope of this paper.

§ 2 Impact on the Standard

Starting with [P0443R11] as a baseline we have the following customization points:

— `execute(executor, invocable) -> void`

— `submit(sender, receiver) -> void`

— `schedule(scheduler) -> sender`

— `set_done`

— `set_error`

— `set_value`

and the following Concepts:

— `executor`

— `scheduler`

— `callback_signal`

— `callback`

— `sender`

First, we should add wording such that the sender algorithms are defined as per range-adapters such that:

— `algorithm(sender, args...)`

— `algorithm(args...)(sender)`

— `sender | algorithm(args...)`

are equivalent. In this way we can use `operator|` in code to make the code more readable.

We propose immediately discussing the addition of the following algorithms:

— `is_noexcept_sender(sender) -> bool`

— `just(T) -> sender`

— `just_error(E) -> sender`

— `via(sender, scheduler) -> sender`

— `sync_wait(sender) -> T`

— `transform(sender, invocable) -> sender`

— `bulk_transform(sender, invocable) -> sender`

— `handle_error(sender, invocable) -> sender`

Details below are in loosely approximated wording and should be made consistent with [P0443R11] and the standard itself when finalized. We choose this set of algorithms as a basic set to allow a range of realistic, though still limited, compositions to be written against executors.

§ 2.1 execution::is_noexcept_sender

§ 2.1.1 Summary

Queries whether the passed sender will ever propagate an error when treated as an r-value to `submit`.

2.1.2 Wording

The name `execution::is_noexcept_sender` denotes a customization point object. The expression `execution::is_noexcept_sender(S)` for some subexpression `S` is expression-equivalent to:

— `S.is_noexcept()`, if that expression is valid.

— Otherwise, `is_noexcept_sender(S)`, if that expression is valid, with overload resolution performed in a context that includes the declaration

```
                template<class S>
                  void is_noexcept_sender(S) = delete;
```

```
        and that does not include a declaration of
`execution::is_noexcept_sender`.
```

— Otherwise, `false`.

If possible, `is_noexcept_sender` should be `noexcept`.

If `execution::is_noexcept_sender(s)` returns true for a `sender` `s` then it is guaranteed that `s` will not call `error` on any `callback` `c` passed to `submit(s, c)`.

2.2 execution::just

2.2.1 Summary

Returns a sender that propagates the passed value inline when `submit` is called. This is useful for starting a chain of work.

2.2.2 Wording

The expression `execution::just(t)` returns a sender, `s` wrapping the value `t`.

— If `t` is nothrow movable then `execution::is_noexcept_sender(s)` shall be constexpr and return true.

— When `execution::submit(s, r)` is called for some `r`, and r-value `s` will call `execution::set_value(r, std::move(t))`, inline with the caller.

— When `execution::submit(s, r)` is called for some `r`, and l-value `s` will call `execution::set_value(r, t)`, inline with the caller.

— If moving of `t` throws, then will catch the exception and call `execution::set_error(r, e)` with the caught `exception_ptr`.

2.3 execution::just_error

2.3.1 Summary

Returns a sender that propagates the passed error inline when `submit` is called. This is useful for starting a chain of work.

2.3.2 Wording

The expression `execution::just_error(e)` returns a sender, `s` wrapping the error `e`.

- If `t` is nothrow movable then `execution::is_noexcept_sender(s)` shall be constexpr and return true.
- When `execution::submit(s, r)` is called for some `r`, and r-value `s` will call `execution::set_error(r, std::move(t))`, inline with the caller.
- When `execution::submit(s, r)` is called for some `r`, and l-value `s` will call `execution::set_error(r, t)`, inline with the caller.
- If moving of `e` throws, then will catch the exception and call `execution::set_error(r, e)` with the caught `exception_ptr`.

2.4 execution::sync_wait

2.4.1 Summary

Blocks the calling thread to wait for the resulting sender to complete. Returns a std::optional of the value or throws if an exception is propagated.[1] On propagation of the `set_done()` signal, returns an empty optional.

2.4.2 Wording

The name `execution::sync_wait` denotes a customization point object. The expression `execution::sync_wait(S)` for some subexpression `S` is expression-equivalent to:

- `S.sync_wait()` if that expression is valid.
- Otherwise, `sync_wait(S)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S>
  void sync_wait(S) = delete;
```

and that does not include a declaration of `execution::sync_wait`.

- Otherwise constructs a `receiver`, `r` over an implementation-defined synchronization primitive and passes that `receiver` to `execution::submit(S, r)`. Waits on the synchronization primitive to block on completion of `S`.
  - If `set_value` is called on `r`, returns a `std::optional` wrapping the passed value.
  - If `set_error` is called on `r`, throws the error value as an exception.

— If `set_done` is called on `r`, returns an empty `std::optional`.

If `execution::is_noexcept_sender(S)` returns true at compile-time, and the return type `T` is nothrow movable, then `sync_wait` is noexcept. Note that `sync_wait` requires `S` to propagate a single value type.

2.5 execution::via

2.5.1 Summary

Transitions execution from one executor to the context of a scheduler. That is that:

```
        sender1 | via(scheduler1) | bulk_execute(f)...
```

will create return a sender that runs in the context of `scheduler1` such that `f` will run on the context of `scheduler1`, potentially customized, but that is not triggered until the completion of `sender1`.

`via(S1, S2)` may be customized on either or both of `S1` and `S2`. For example any two senders with their own implementations may provide some mechanism for interoperation that is more efficient than falling back to simple callbacks.

2.5.2 Wording

The name `execution::via` denotes a customization point object. The expression `execution::via(S1, S2)` for some subexpressions `S1`, `S2` is expression-equivalent to:

— `S1.via(S2)` if that expression is valid.

— Otherwise, `via(S1, S2)` if that expression is valid with overload resolution performed in a context that includes the declaration

```
                template<class S1, class S2>
                  void via(S1, S2) = delete;
```

— Otherwise constructs a receiver `r` such that when `set_value`, `set_error` or `set_done` is called on `r` the value(s) or error(s) are packaged, and a receiver `r2` constructed such that when `execution::set_value(r2)` is called, the stored value or error is transmitted and `r2` is submitted to `S2`.

— The returned sender's value types match those of `sender1`.

— The returned sender's execution context is that of `scheduler1`.

If `execution::is_noexcept_sender(S1)` returns true at compile-time, and `execution::is_noexcept_sender(S2)` returns true at compile-time and all entries in `S1::value_types` are nothrow movable, `execution::is_noexcept_sender(on(S1, S2))` should return `true` at compile time[2].

2.6 execution::transform

2.6.1 Summary

Applies a function `f` to the value channel of a sender such that some type list `T...` is consumed and some type `T2` returned, resulting in a sender that transmits `T2` in its value channel. This is equivalent to common `Future::then` operations, for example:

```
Future<float>f = folly::makeFuture<int>(3).thenValue(
  [](int input){return float(input);});
```

2.6.2 Wording

The name `execution::transform` denotes a customization point object. The expression `execution::transform(S, F)` for some subexpressions `S` and `F` is expression-equivalent to:

— `S.transform(F)` if that expression is valid.

— Otherwise, `transform(S, F)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S, class F>
  void transform(S, F) = delete;
```

and that does not include a declaration of `execution::transform`.

— Otherwise constructs a receiver, `r` over an implementation-defined synchronization primitive and passes that receiver to `execution::submit(S, r)`. When some `output_receiver` has been passed to `submit` on the returned `sender`.

  — If `set_value(r, Ts... ts)` is called, calls `std::invoke(F, ts...)` and passes the result `v` to `execution::set_value(output_receiver, v)`.

  — If `F` throws, catches the exception and passes it to `execution::set_error(output_receiver, e)`.

  — If `set_error(c, e)` is called, passes `e` to `execution::set_error(output_receiver, e)`.

  — If `set_done(c)` is called, calls `execution::set_done(output_receiver)`.

If `execution::is_noexcept_sender(S)` returns true at compile-time, and `F(S1::value_types)` is marked `noexcept` and all entries in `S1::value_types` are nothrow movable, `execution::is_noexcept_sender(transform(S1, F))` should return `true` at compile time.

2.7 execution::bulk_transform

2.7.1 Summary

`bulk_execute` is a side-effecting operation across an iteration space. `bulk_transform` is a very similar operation that operates element-wise over an input range and returns the

result as an output range of the same size.

2.7.2 Wording

The name `execution::bulk_transform` denotes a customization point object. The expression `execution::bulk_transform(S, F)` for some subexpressions S and F is expression-equivalent to:

— S.bulk_transform(F), if that expression is valid.

— Otherwise, `bulk_transform(S, F)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S, class F>
  void bulk_transform(S, F) = delete;
```

and that does not include a declaration of `execution::bulk_transform`.

— Otherwise constructs a receiver, `r` over an implementation-defined synchronization primitive and passes that receiver to `execution::submit(S, r)`.

  — If `S::value_type` does not model the concept `Range<T>` for some `T` the expression ill-formed.

  — If `set_value` is called on `r` with some parameter `input` applies the equivalent of `out = std::ranges::transform_view(input, F)` and passes the result `output` to `execution::set_value(output_receiver, v)`.

  — If `set_error(r, e)` is called, passes `e` to `execution::set_error(output_receiver, e)`.

  — If `set_done(r)` is called, calls `execution::set_done(output_receiver)`.

2.8 execution::handle_error

2.8.1 Summary

This is the only algorithm that deals with an incoming signal on the error channel of the `sender`. Others only deal with the value channel directly. For full generality, the formulation we suggest here applies a function `f(e)` to the error `e`, and returns a `sender` that may output on any of its channels. In that way we can solve and replace an error, cancel on error, or log and propagate the error, all within the same algorithm.

2.8.2 Wording

The name `execution::handle_error` denotes a customization point object. The expression `execution::handle_error(S, F)` for some subexpressions S and F is expression-equivalent to:

— S.handle_error(F), if that expression is valid.

— Otherwise, `handle_error(S, F)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S, class F>
  void handle_error(S, F) = delete;
```

and that does not include a declaration of `execution::handle_error`.

— Otherwise constructs a receiver, `r` over an implementation-defined synchronization primitive and passes that receiver to `execution::submit(S, r)`.

  — If `set_value(r, v)` is called, passes `v` to `execution::set_value(output_receiver, v)`.

  — If `set_error(r, e)` is called, passes `e` to `f`, resulting in a `sender` `s2` and passes `output_receiver` to `submit(s2, output_receiver)`.

  — If `set_done(r)` is called, calls `execution::set_done(output_receiver)`.

§ 3 Customization and example

Each of these algorithms, apart from `just`, is customizable on one or more `sender` implementations. This allows full optimization. For example, in the following simple work chain:

```
auto s = just(3) |                                          // s1
         via(scheduler1) |                                  // s2
         transform([](int a){return a+1;}) |                // s3
         transform([](int a){return a*2;}) |                // s4
         on(scheduler2) |                                   // s5
         handle_error([](auto e){return just_error(e);}); // s6
int r = sync_wait(s);
```

The result of `s1` might be a `just_sender<int>` implemented by the standard library vendor.

`on(just_sender<int>, scheduler1)` has no customization defined, and this expression returns an `scheduler1_on_sender<int>` that is a custom type from the author of `scheduler1`, it will call `submit` on the result of `s1`.

`s3` calls `transform(scheduler1_on_sender<int>, [](int a){return a+1;})` for which the author of `scheduler1` may have written a customization. The `scheduler1_on_sender` has stashed the value somewhere and build some work queue in the background. We do not see `submit` called at this point, it uses a behind-the-scenes implementation to schedule the work on the work queue. An `scheduler1_transform_sender<int>` is returned.

`s4` implements a very similar customization, and again does not call `submit`. There need be no synchronization in this chain.

At `s5`, however, the implementor of `scheduler2` does not know about the implementation of `scheduler1`. At this point it will call `submit` on the incoming `scheduler1_transform_sender`, forcing `scheduler1`'s sender to implement the necessary synchronization to map back from the behind-the-scenes optimal queue to something interoperable with another vendor's implementation.

`handle_error` at `s6` will be generic in terms of `submit` and not do anything special, this uses the default implementation in terms of `submit`. `sync_wait` similarly constructs a `condition_variable` and a temporary `int`, submits a `receiver` to `s` and waits on the `condition_variable`, blocking the calling thread.

`r` is of course the value 8 at this point assuming that neither scheduler triggered an error.

§ 4 References

[P0443R11] 2019. A Unified Executors Proposal for C++.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0443r11.html

[P1660R0] 2019. A Compromise Executor Design Sketch.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1660r0.pdf

---

1. Conversion from asynchronous callbacks to synchronous function-return can be achieved in different ways. A `CancellationException` would be an alternative approach.↩

2. Should, shall, may?↩